Open Source Architecture Appendix

Chen Xingqiang · github.com/chenxingqiang · one project overview and architecture per page

Primary flow
Fallback / feedback
Dashed = architecture zone
Box = module + responsibility

CONTENTS · one architecture per page

TraceSmith
GPU profiling & replay

YiRage
RuntimeFusion compiler

JAX-sklearn
transparent JAX acceleration

Secret-Learn
privacy-preserving ML

YouTube-DNN
two-tower recall

Research CLI
agent workflow

Questions-Gen
three-stage training

READING GUIDE

Read top to bottom. Dashed containers are architecture zones; boxes are modules and responsibilities.

Solid blue arrows show the primary path; dashed purple arrows show fallback or feedback paths.

TraceSmith — 系统架构 · GPU Trace 端到端数据流

Application → bindings → capture / state / format → replay & export → GPU backends

Primary flow
Fallback / feedback
Dashed = architecture zone
Box = module + responsibility

PROJECT OVERVIEW TraceSmith is a cross-platform GPU profiling and deterministic replay system for AI compiler, framework, and driver engineers.
Its C++17 core captures low-overhead execution events, stores compact SBT traces, and exports timelines for analysis and debugging.

APPLICATION · 用户入口

- **C++ / Python Application** — 嵌入业务进程
- **CLI** — profile · record · view · export
- **External UI** — Perfetto · Chrome

BINDINGS · Python / C++ 边界

- **_tracesmith.so** — pybind11 native extension

CORE LIBRARY · C++17

- **Capture Module** — CUPTIProfiler · MCPTIProfiler; Metal · Ascend · ROCm · eBPF
- **Common + Format** — RingBuffer SPSC · StackCapture; SBTWriter / SBTReader
- **State Module** — GPUStateMachine; TimelineBuilder · PerfettoExporter
- **Replay Module** — ReplayEngine · StreamScheduler; DeterminismChecker · FrameCapture

CROSS-CUTTING · 集群与调试

replay / debug feedback

- **Cluster Module** — GPUTopology · TimeSync; NCCLTracker
- **GDB / Tracy** — RSPHandler · GPUDebugEngine; TracyExporter

GPU BACKENDS · 系统接口

- **NVIDIA** — CUDA · CUPTI · nsys
- **MetaX** — MACA · MCPTI · mcTracer
- **Apple / AMD** — Metal · xctrace · ROCm
- **Huawei / Linux** — CANN · msprof · eBPF

OUTPUTS · Trace 产物

- **trace.sbt** — compact binary
- **trace.json / .perfetto** — Perfetto timeline
- **graph.dot / ASCII** — analysis views

主链：GPU event → lock-free ring → SBT/state → replay 或 Perfetto；Cluster / GDB 为旁路能力。

YiRage (YPK) — 系统架构 · RuntimeFusion 数据流

Inference engine owns scheduling/KV; RF owns fusion identity; search and execute use the same backend

Primary flow
Fallback / feedback
Dashed = architecture zone
Box = module + responsibility

PROJECT OVERVIEW YiRage is a multi-backend LLM kernel compiler and runtime centered on graph superoptimization and RuntimeFusion. It searches backend-specific fusion plans offline, then injects verified FusionCapsules into vLLM or SGLang decode steps.

INFERENCE ENGINE · 调度与 KV 所有者

- **vLLM** — continuous batch · PagedAttention
- **SGLang** — RadixAttention · extend meta
- **Decode Metadata** — block table · seq lens · SM budget

RUNTIMEFUSION · 每个 decode step

- **RF.step(inputs, meta)** — select 0..N capsules
- **FusionCapsule** — MLP gate_up + down segment pipeline
- **Native Fallback** — skip → engine native ops

SEARCH & PLAN · offline / warm-up

- **superoptimize(backend)** — KN / TB graph search
- **FusionPlan / μGraph** — profile · fingerprint verify
- **Persistent Cache** — shape bucket restore
- **Ray Coordinator** — optional distributed search — fallback / re-search

BACKEND MANAGER · 编译与执行边界

- **BackendRegistry** — search + execute same backend
- **KNGraph / TBGraph** — transpiler → kernels
- **yirage.core** — Cython runtime

BACKENDS → HARDWARE

- **CUDA / MACA** — NVIDIA GPU · MetaX C500/C550
- **ROCm / MPS** — AMD GPU · Apple Silicon
- **Ascend / NKI** — Huawei 910 · AWS Neuron
- **CPU / MLIR / Triton** — x86 · ARM · extra targets

主链：Engine metadata → RF.step → FusionCapsule → BackendRegistry → target hardware。

JAX-sklearn (xlearn) — 五层架构 · 透明加速数据流

100% sklearn API → runtime injection → proxy dispatch → array conversion → JAX hardware

Primary flow
Fallback / feedback
Dashed = architecture zone
Box = module + responsibility

PROJECT OVERVIEW JAX-sklearn provides a drop-in scikit-learn API while transparently accelerating supported estimators with JAX.
Runtime proxies select JAX or native sklearn by algorithm, data size, and platform, preserving familiar inputs and outputs.

USER CODE · sklearn-compatible API

import xlearn as sklearn
LinearRegression · KMeans · PCA

Estimator Calls
fit · predict · transform

COMPATIBILITY · xlearn/__init__.py

__getattr__
lazy module loading

_auto_jax_accelerate_module
runtime class injection

AcceleratorRegistry
algorithm implementation map

JAX ACCELERATION · xlearn/_jax

EstimatorProxy
fit-time dispatch

Universal JAX
JIT · vmap · accelerated estimator

sklearn Fallback
original estimator
graceful fallback

DATA MANAGEMENT · boundary conversion

NumPy → JAX
dtype · shape normalization

Policy / Thresholds
dataset size · platform · config

JAX → NumPy
sklearn-compatible outputs

HARDWARE ABSTRACTION · JAX backend

CPU
jaxlib

CUDA GPU
JAX XLA

TPU
JAX XLA

Apple Metal
experimental

代理根据算法、数据规模和配置选择 JAX；不满足条件或执行失败时回退原生 sklearn。

Secret-Learn — 六层架构 · 隐私计算数据流

sklearn-compatible API → privacy mode → JAX / SecretFlow compute → secure devices

Primary flow
Fallback / feedback
Dashed = architecture zone
Box = module + responsibility

PROJECT OVERVIEW Secret-Learn exposes sklearn-compatible machine learning across federated, secret-sharing, and split-learning modes.
It combines JAX acceleration with SecretFlow devices so each workflow can choose where plaintext, shares, and activations reside.

APPLICATION · 多方隐私数据

- **Healthcare / Finance** — cross-party modeling
- **IoT / Research** — distributed learning

SKLEARN API · 统一算法入口

- **secretlearn.*** — fit · predict · transform
- **algorithm_classifier.py** — classify fit signature / epochs

PRIVACY MODES · execution placement

- **FEDERATED LEARNING** — PYU local compute; HEU secure aggregation
- **SECRET SHARING** — SPU MPC · ABY3 / CHEETAH
- **SPLIT LEARNING** — split model · encrypted activations

ACCELERATION & COMPUTE

- **JAX Acceleration** — JIT · vmap · xlearn
- **SecretFlow Runtime** — party orchestration

SECURE DEVICES

- **PYU** — party-local plaintext
- **HEU** — homomorphic aggregation
- **SPU** — secure MPC
- **TEE** — optional enclave

INFRASTRUCTURE & HARDWARE

- **SecretFlow** — distributed secure runtime
- **JAX / XLA** — CPU · GPU · TPU
- **sklearn** — API compatibility

同一 sklearn 调用由 FL / SS / SL 决定数据驻留与执行设备；JAX 加速可公开的本地计算。

YouTube-DNN-RecSys — 系统架构 · 召回数据流

Python / Spark data preparation → TFRecords → dnn.py → user & item vectors → prediction

Primary flow
Fallback / feedback
Dashed = architecture zone
Box = module + responsibility

PROJECT OVERVIEW YouTube-DNN-RecSys is a TensorFlow and Spark implementation of the candidate-generation stage of YouTube recommendations. It turns behavior histories into TFRecords, learns user and item embeddings, and produces Top-K recall candidates for serving.

DATA PROCESSING · 离线样本构建

Python Pipeline
data2tfrecords.py
read_tfrecords.py

Spark Pipeline
MakeDataOne.scala
MakeDataTwo.scala

FeatureBuilder
history + continuous features

STORAGE · 训练与模型产物

TFRecords
train / evaluation

Checkpoints
ckpt / eval / modelpath

MODEL TRAINING · src/python/models/dnn.py

Input
visit_items_index × 5
continuous_features × 16

Item Embedding
500 × 16 · average pooling

User Tower
Dense 64 → 32 → vector 16

Softmax Output
500 classes · Top-K

evaluation feedback

PREDICTION · serving paths

Python SavedModel
load_dnn_model.py

User Vector
PredictUserVector.scala

Item Table
ItemEmbeddingPredictor.scala

RECOMMENDATION OUTPUT

Candidate Recall
user vector × item embedding

Top-K Items
next-item recommendation

主链：行为历史 → TFRecords → embedding mean-pool → user vector → item table matching → Top-K。

Research CLI — 系统架构 · Agent 请求数据流

Terminal UI → core orchestration & routing → tools / MCP / research modules → model providers

Primary flow
Fallback / feedback
Dashed = architecture zone
Box = module + responsibility

PROJECT OVERVIEW Research CLI is a terminal-first AI agent workflow that connects interactive sessions to models, tools, and MCP services.
Its core router manages context and provider selection while research modules cover writing, bibliography, analysis, and submission.

CLI FRONTEND · packages/cli

- **Interactive TUI** — research.tsx · sessions
- **Commands** — mcp · skills · hooks
- **ACP** — acpSession.ts

CORE ORCHESTRATION · packages/core

- **Context & Memory** — config.ts · memory.ts
- **modelRouterService** — provider selection
- **Routing Strategy** — default · classifier · approval

TOOLS & INTEGRATIONS

- **Workspace Tools** — filesystem · shell
- **MCP** — external service adapters
- **Research Modules** — writing · bibliography · analysis
- **SDK / A2A / VS Code** — embedded clients

tool result loop

MODEL PROVIDERS · provider config

- **Gemini / Vertex** — hosted models
- **OpenAI / Anthropic** — hosted models
- **Baidu / DeepSeek / Qwen** — hosted models

RESULT STREAM

- **Tool Results** — structured observations
- **Assistant Response** — streamed to terminal

主链：prompt → packages/cli → packages/core router → provider / tools → structured result → terminal。

Questions-Gen — 系统架构 · 三阶段训练数据流

Training data → SFT → GRPO → DeepSeek distillation → validation → Hugging Face / Ollama

Primary flow
Fallback / feedback
Dashed = architecture zone
Box = module + responsibility

PROJECT OVERVIEW Questions-Gen is a Qwen3-14B system for generating and evaluating mathematical competition problems through staged training. The pipeline combines supervised fine-tuning, GRPO reward optimization, DeepSeek-R1 distillation, validation, and local deployment.

TRAINING DATA · problem mixture

- **Historical Problems** — 50% source distribution
- **Variants** — 30% conditional variation
- **Novel Problems** — 20% innovation target

Stage 1 SFT · QuestionsGenTrainer.py

- **Qwen3-14B** — supervised fine-tuning
- **Stage-1 Checkpoint** — stage1-fp-merged

Stage 2 GRPO · reward optimization

- **RewardCalculator** — 0.4 difficulty · 0.3 novelty · 0.2 rigor · 0.1 diversity
- **NoveltyConstraint** — variation / duplication guard
- **GRPO Trainer** — group-relative optimization

Stage 3 Distill · teacher supervision

quality feedback

- **DeepSeekTeacher** — DeepSeek-R1 reasoning teacher
- **Final Model** — final-merged-16bit

VALIDATION

- **ModelValidator** — single checkpoint
- **BatchValidator** — algebra · geometry · calculus
- **QualityEvaluator** — clarity · difficulty · originality

DEPLOYMENT

- **Hugging Face** — three published stages
- **Ollama** — local inference
- **questions-gen CLI** — validate · batch · quality

主链：混合题库 → SFT → GRPO reward/novelty → DeepSeek-R1 distill → validate → publish。